Java Collections — Tổng hợp kiến thức

1. Cây phân cấp

```
Iterable
 └── Collection
      ├── List   có thứ tự, CHO phép trùng    : ArrayList, LinkedList
      ├── Set    KHÔNG cho trùng              : HashSet, LinkedHashSet, TreeSet
      └── Queue  hàng đợi                     : ArrayDeque, PriorityQueue

Map (KHÔNG thuộc Collection) cặp key-value    : HashMap, LinkedHashMap, TreeMap
```

Ba điều hay bị hỏi:

- `Collection` là interface, `Collections` (có s) là class tiện ích toàn hàm static
- `Map` **không** kế thừa `Collection`
- Collection chỉ chứa **object**: `List<Integer>`, không phải `List<int>`

Bảng kiểu bọc: `int→Integer`, `double→Double`, `char→Character`, `boolean→Boolean`, `long→Long`

2. Chọn cấu trúc nào

Nhu cầu	Dùng
Danh sách, truy cập theo index	`ArrayList`
Loại trùng	`HashSet`
Loại trùng + giữ thứ tự thêm vào	`LinkedHashSet`
Loại trùng + tự sắp xếp	`TreeSet`
Tra cứu theo key, đếm tần suất	`HashMap`
Map giữ thứ tự put	`LinkedHashMap`
Map có key sắp xếp	`TreeMap`
Stack / Queue	`ArrayDeque`

Ba họ Hash / LinkedHash / Tree hoạt động **giống hệt nhau** ở Set và Map — chỉ khác thứ tự.

Tiền tố	Thứ tự
`Hash...`	không đảm bảo
`LinkedHash...`	theo thứ tự thêm vào
`Tree...`	sắp xếp tăng dần

`HashSet`/`HashMap` đôi khi *trông như* đã sắp xếp — đó là trùng hợp, không được dựa vào.

---

3. List

```java
List<String> list = new ArrayList<>();

list.add("a");              // thêm cuối
list.add(1, "b");           // chèn vào index 1
list.get(0);                // đọc theo index
list.set(0, "x");           // sửa
list.remove("a");           // xoá theo GIÁ TRỊ
list.remove(0);             // xoá theo INDEX
list.size();
list.contains("a");
list.indexOf("a");
list.isEmpty();
list.removeIf(s -> s.isEmpty());

for (String s : list) { }                    // duyệt theo giá trị
for (int i = 0; i < list.size(); i++) { }    // duyệt theo index
```

Khác mảng: mảng dùng `a.length` và `a[i]`; List dùng `size()`, `get(i)`, `set(i, x)`.

`remove` với `List<Integer>`:

```java
nums.remove(1);                    // xoá INDEX 1
nums.remove(Integer.valueOf(1));   // xoá GIÁ TRỊ 1
```

Xoá khi đang duyệt:

```java
for (Integer n : nums) nums.remove(n);    // ❌ ConcurrentModificationException
nums.removeIf(n -> n < 0);                // ✅
```

`List.of()` **là bất biến** — muốn sửa phải bọc: `new ArrayList<>(List.of(...))`

ArrayList vs LinkedList: mặc định dùng **ArrayList**. LinkedList chỉ hơn khi thêm/xoá liên tục ở hai đầu — mà trường hợp đó `ArrayDeque` còn tốt hơn.

---

4. Set

```java
Set<String> set = new HashSet<>();

set.add("a");               // trả về FALSE nếu đã có → dùng để phát hiện trùng
set.contains("a");          // nhanh hơn list.contains rất nhiều
set.remove("a");
set.size();

for (String s : set) { }    // KHÔNG có get(i)
```

Mẫu phát hiện trùng:

```java
if (!seen.add(x)) { /* x đã gặp trước đó */ }
```

Phép tập hợp — các hàm này **sửa trực tiếp**, luôn copy trước:

```java
Set<Integer> r = new HashSet<>(a);
r.addAll(b);       // hợp
r.retainAll(b);    // giao
r.removeAll(b);    // hiệu a - b
```

`TreeSet` có thêm: `first()`, `last()`

---

5. Map

```java
Map<String, Integer> map = new HashMap<>();
//      ↑ key    ↑ value

map.put("a", 1);                    // thêm HOẶC ghi đè (không báo lỗi)
map.get("a");                       // null nếu không có
map.getOrDefault("a", 0);           // an toàn hơn
map.containsKey("a");
map.containsValue(1);
map.remove("a");
map.size();

for (String k : map.keySet()) { }                       // chỉ key
for (Integer v : map.values()) { }                      // chỉ value
for (Map.Entry<String, Integer> e : map.entrySet())     // cả hai
    { e.getKey(); e.getValue(); }
```

Mẫu đếm tần suất — quan trọng nhất:

```java
map.put(x, map.getOrDefault(x, 0) + 1);
// đọc số cũ (chưa có → 0) → cộng 1 → ghi lại
```

Mẫu nhóm (value là List):

```java
if (!map.containsKey(k)) map.put(k, new ArrayList<>());
map.get(k).add(value);
```

Bẫy NullPointerException:

```java
int n = map.get("z");          // ❌ get trả về null, không ép sang int được
int n = map.getOrDefault("z", 0);   // ✅
```

`TreeMap` có thêm: `firstKey()`, `lastKey()`

---

6. Sắp xếp

```java
Collections.sort(list);                          // tăng dần, SỬA list gốc
Collections.reverse(list);
list.sort(Collections.reverseOrder());           // giảm dần
Collections.max(list);  Collections.min(list);
```

`sort` không trả về gì. Muốn giữ list gốc thì copy trước: `new ArrayList<>(list)`

Thứ tự tự nhiên của `String` dựa trên mã ký tự → **chữ hoa đứng trước chữ thường** (`"Zebra"` trước `"apple"`).

Comparable — thứ tự mặc định, viết trong class

```java
class Student implements Comparable<Student> {
    public int compareTo(Student o) {
        return Integer.compare(this.id, o.id);
    }
}
```

Comparator — thứ tự tuỳ biến, viết lúc gọi sort

```java
list.sort((a, b) -> Integer.compare(a.id, b.id));        // tăng
list.sort((a, b) -> Integer.compare(b.id, a.id));        // giảm (đảo a, b)
list.sort((a, b) -> a.name.compareTo(b.name));           // chuỗi A→Z
list.sort((a, b) -> b.name.compareTo(a.name));           // chuỗi Z→A
```

`a` và `b` do `sort` tự truyền vào, mỗi lần một cặp phần tử khác nhau.

Quy ước trả về: âm = `a` đứng trước, 0 = bằng nhau, dương = `a` đứng sau.

	Comparable	Comparator
Viết ở đâu	trong class	lúc gọi sort
Số thứ tự	chỉ 1	bao nhiêu cũng được
Hàm	`compareTo(other)`	`compare(a, b)`

Nhiều tiêu chí

java

```
list.sort((a, b) -> {
    int r = Double.compare(b.cgpa, a.cgpa);   // tiêu chí 1 (giảm dần)
    if (r != 0) return r;                      // khác nhau → xong
    return a.name.compareTo(b.name);           // bằng nhau → tiêu chí 2
});
```

Cách viết ngắn (Java 8+):

java

```
list.sort(Comparator.comparingDouble((Student s) -> s.cgpa).reversed()
          .thenComparing(s -> s.name));
```

So sánh an toàn

java

```
Integer.compare(a, b)     // thay cho a - b (tránh tràn số)
Double.compare(a, b)      // BẮT BUỘC với double
a.name.compareTo(b.name) // chuỗi
```

Không dùng `(int)(a.cgpa - b.cgpa)` — `3.9 - 3.5 = 0.4` ép sang int thành 0.

Sắp Map theo value

java

```
List<Map.Entry<String,Integer>> es = new ArrayList<>(map.entrySet());
es.sort((a, b) -> Integer.compare(b.getValue(), a.getValue()));
```

7. toString, equals & hashCode

toString — quyết định object hiện ra thế nào khi in

```java
@Override
public String toString() { return id + " " + name + " " + cgpa; }
```

Java tự gọi khi `println(obj)`, nối chuỗi, in cả List. Không có thì in ra `Student@1b6d3586`.

equals & hashCode — bắt buộc khi class vào Set hoặc làm key Map

```java
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Student s = (Student) o;
    return id == s.id;
}

@Override
public int hashCode() { return Objects.hash(id); }
```

Mặc định Java so sánh object bằng **địa chỉ bộ nhớ**, nên hai object cùng nội dung vẫn bị coi là khác nhau.

Quy tắc: override `equals` thì **phải** override `hashCode`, dùng **cùng bộ field**.

Lý do: HashSet/HashMap tìm qua 2 bước — `hashCode()` chọn "ngăn", rồi `equals()` tìm trong ngăn đó. Thiếu `hashCode` thì hai object giống nhau rơi vào hai ngăn khác nhau, `equals()` không bao giờ được gọi.

`@Override` không bắt buộc, nhưng giúp compiler bắt lỗi gõ sai tên hàm.

== vs equals

```java
a == b          // so ĐỊA CHỈ — chỉ dùng cho int, double, char, boolean
a.equals(b)     // so NỘI DUNG — dùng cho mọi object
```

---

8. Stack & Queue (ArrayDeque)

```java
Deque<Integer> stack = new ArrayDeque<>();
stack.push(x);   stack.pop();   stack.peek();    // vào sau ra trước

Deque<Integer> queue = new ArrayDeque<>();
queue.offer(x);  queue.poll();  queue.peek();    // vào trước ra trước

while (!stack.isEmpty()) { stack.pop(); }
```

	Stack	Queue
Thêm	`push` (vào đầu)	`offer` (vào cuối)
Lấy	`pop`	`poll`
Xem không lấy	`peek`	`peek`

Khác biệt duy nhất là **thêm vào đâu**; cả hai đều lấy ra từ đầu.

Luôn `isEmpty()` trước khi `pop`/`poll` — gọi trên deque rỗng sẽ ném `NoSuchElementException`.

Không dùng class `Stack` cũ — chậm và lỗi thời, tài liệu Java khuyên dùng `ArrayDeque`.

Mẫu kiểm tra ngoặc cân bằng:

```
ngoặc mở  → push
ngoặc đóng → nếu stack rỗng: false
             pop ra, không khớp: false
hết chuỗi → return stack.isEmpty()
```

---

9. Chuyển đổi & hàm phụ trợ

```java
```

```
new HashSet<>(list);                // List → Set (loại trùng)
new ArrayList<>(set);               // Set → List
new TreeSet<>(list);                // loại trùng + sắp xếp
new ArrayList<>(List.of(1,2,3));    // tạo list SỬA ĐƯỢC
Arrays.asList(array);               // mảng → List

sentence.split(" ");                // chuỗi → mảng từ
s.charAt(i);                        // ký tự thứ i (char)
s.substring(i, j);                  // đoạn con (String)
s.toUpperCase();                    // PHẢI hứng kết quả — String bất biến
```

Ba kiểu "độ dài":

Đối tượng	Cách lấy
Mảng `int[] a`	`a.length`
Chuỗi `String s`	`s.length()`
Collection	`.size()`

10. Bẫy thường gặp

Bẫy	Sai	Đúng
Kiểu nguyên thuỷ trong generic	`List<int>`	`List<Integer>`
Vế trái về phải khác họ	`Set x = new ArrayList<>()`	`Set x = new HashSet<>()`
Map thiếu kiểu	`Map<Integer> m`	`Map<Integer, String> m`
Khởi tạo "sổ ghi nhớ" kèm dữ liệu	`new HashSet<>(input)`	`new HashSet<>()`
Điều kiện lọc dùng cả tập hợp	`if (list.size() > 3)`	`if (s.length() > 3)`
For-each rồi vẫn `get()`	`for (int n : nums) nums.get(n)`	`for (int n : nums)` — n đã là giá trị
Biến tích luỹ trong vòng lặp	`for(...) { int max = ...; }`	khai báo **trước** vòng lặp
`return` phủ định trong vòng lặp	`else return false;`	đặt **sau** vòng lặp
`sort` trả về list mới	`List x = Collections.sort(l)`	`Collections.sort(l)` — sửa tại chỗ
So double bằng phép trừ	`(int)(a - b)`	`Double.compare(a, b)`
Quên `!=0` khi nhiều tiêu chí	luôn `return r`	`if (r != 0) return r;`
`get` trả null rồi ép int	`int n = map.get(k)`	`map.getOrDefault(k, 0)`
`pop` khi chưa check rỗng	`stack.pop()`	`if (!stack.isEmpty())` trước

Hai nguyên tắc chung rút ra từ bảng trên:

1. **Biến cần nhớ qua nhiều vòng lặp** (max, biến đếm, Set, Map, stack, chuỗi tích luỹ) luôn khai báo **trước** vòng lặp.
2. **Kết luận khẳng định** (tìm thấy) có thể `return` trong vòng lặp. **Kết luận phủ định** (không tìm thấy, lớn nhất, không trùng) phải `return` **sau** vòng lặp.

---

11. Bảng tra nhanh

Việc	Code
Loại trùng giữ thứ tự	`new LinkedHashSet<>(list)`
Loại trùng + sắp xếp	`new TreeSet<>(list)`
Phát hiện trùng	`if (!seen.add(x))`
Đếm tần suất	`map.put(x, map.getOrDefault(x,0)+1)`
Duyệt Map cả key lẫn value	`for (Map.Entry<K,V> e : map.entrySet())`
Nhóm theo khoá	`Map<K, List<V>>` + tạo list rỗng khi chưa có
Sắp giảm dần	đảo `a`, `b` trong Comparator
Sắp nhiều tiêu chí	`if (r != 0) return r;` rồi tiêu chí sau
Sắp Map theo value	đổ `entrySet()` ra List rồi sort
Đảo chuỗi	push từng ký tự rồi pop hết
Kiểm tra ngoặc	stack + `return stack.isEmpty()`